MOON-2389: AI Labs answers to the AI team topics

Oct 30, 2026 · @Someone

AI Labs answers the open questions in the MOON-2389 Client Operating Model design ("AI team: topics to discuss", pages 39–47; open questions #5, #14, #23 and #24) and the actions from the 30 Oct working session with the Veda Accelerate team. For December we ask for: two or three priority triggers delivered as documented events in one durable feed; a context endpoint with team roles and methodology content; an interim methodology file in the final format; a suggestion endpoint that accepts several target users and reports outcomes; and scoped credentials issued by Veda's key management. Every December action is a suggestion a person approves.

Each section lists the question, then our answer. Final catalog keys follow once the December use cases are agreed. The last section lists what to close at the joint working session.

Topic 1: Which events reach the orchestrator, and how?

Question: Which two or three triggers do we start with in December, how does the orchestrator receive each one, and do we need one consolidated event layer for the MVP?

Answer: We propose three December triggers, plus a fourth only if a December use case needs it. The final choice follows the December use cases (frozen 4 Nov).

Event	Where the signal is today	Identifiers in the payload	Time and history	Delivery
`callpilot.call.ended` (a call completed)	CallPilot; sent straight to WorkflowService as a Celery task today	tenant, project ID, call ID	`occurred_at`; transcript and summary read by call ID	Event feed
`project.health.status_changed` (for example, health turns red)	Project health webhook events in Integration Sync	tenant, project ID, from and to status	`occurred_at`; the previous status in the payload	Event feed
`project.methodology.stage_changed`	New project methodology table	tenant, project ID, methodology version ID, from and to stage ID, who moved it (person or applied suggestion, with `suggestion_id`)	`occurred_at`	Event feed

Event	Where the signal is today	Identifiers in the payload	Time and history	Delivery
`schedule.daily_check` (only if needed, for example chasing pre-work before training)	Veda's scheduler	tenant, project ID	the scheduled date, in the tenant's time zone	Event feed

Later candidates: a user question or request, a delegated agent action completed, and a task completed or changed.

One event layer: yes, for the selected triggers. Each one should reach us through the same durable, replayable feed: MOON-2389's append-only table, written in the same transaction as the change. Two delivery paths with different replay and deduplication rules are the main risk. If a signal can't move into the feed by December, it needs an equally durable alternative.

What every event carries: a documented, stable name (`<domain>.<entity>.<event>`), a payload version, a sequence number, `occurred_at` , and IDs and references only, no message or transcript text. We never infer meaning from raw application tables or discover events at runtime.

On our side:

- We own the feed consumer and its cursor. We save the cursor with our own record of what we processed, and the suggestion deduplication key makes a replay after a crash harmless.
- If one event leads to several suggestions, we move past it only once every item is created or recorded as rejected for good.

We need:

1. The feed, and the selected triggers in it, in the first version.
2. Feed rows kept append-only for the first version, with minimal sensitive fields; long-term retention agreed before indefinite storage is promised.
3. Confidential projects left out of the feed and the context reads, and an agreed behavior when a project's confidentiality changes.
4. A test feed with synthetic data in a dev tenant.

Topic 2: What does the orchestrator need to read from Veda?

Question: What is the minimum context the orchestrator needs for the MVP, and how does the context integration work (schemas, authentication, errors, retries, idempotency, observability)?

Answer: One context endpoint per project, plus the methodology-version endpoint, covers the MVP. The minimum context:

1. **Project:** tenant, project ID, name, status and health, key dates (start, go-live, milestones, training).
2. **Team:** for each member, user ID, name, project role or roles, access level, owner flag, and assignment metadata (date added; allocation if available). We use this to choose who receives a suggestion (topic 4).
3. **Methodology:** the pinned version, the current stage, the stage's activities, the policies that apply (stage-linked and standalone) as `{policy_id, revision}` or resolved content, and a separate instructions field (open question #7). Instructions are client-written, so we treat them as untrusted input, never as commands to our system.
4. **Suggestion history:** what we already sent on the project and where each one stands, so we don't repeat ourselves.
5. **Signals:** whether Slack or Teams is connected. Certinia data (opportunity stage, RAID) keeps coming through Integration Sync's existing Certinia context endpoint.

How the integration works:

- **Schemas:** request and response shapes published as JSON Schema, with a version (topic 5); we validate against them in CI.
- **Authentication:** as in Technical details.
- **Errors:** `{detail, code, meta, request_id}`, with stable error codes.
- **Retries:** reads are idempotent; we retry timeouts and 5xx responses with backoff, never 4xx.
- **Caching:** a 60-second cache per project, cleared by that project's events, is fine for us.
- **Observability:** we send `X-Request-Id` on every call and both sides log it.

Methodology input and the interim file

Question: Will methodology arrive as a text file, structured data or both? Who converts uploaded documents, and how is methodology content retrieved and updated? Veda Accelerate will share an interim file within days so integration can start before the UI and

backend are finished.

Answer: Structured data is the contract, with the natural-language parts carried inside it.

- **Format:** JSON in the same shape the methodology product will publish: the methodology version snapshot in MOON-2389 (methodology, stages, items of kind activity or policy, policy triggers, roles, agents), plus standalone policies and the project or practice each applies to. Policy instructions, activity descriptions and prompts are text fields inside it.
- **Interim file:** one JSON file per methodology in exactly that shape, with a `schema_version`. We build against it now; the production endpoint replaces it with no change on our side beyond the URL. Please include at least one methodology with a standalone policy (for example, health turns red) and one activity with an explicit agent or action.
- **Retrieval:** through the context endpoint and the methodology-version endpoint (topic 2). For the interim file, a shared location is fine until the endpoint exists.
- **Updates:** clients change methodology through Veda's product (upload, then incremental edits). We don't write methodology in the first version; later, any change we propose goes through Veda's proposals flow for an admin to accept.
- **Conversion of uploaded documents:** we propose Veda owns it, as part of the upload experience, producing the same JSON. If Veda wants AI Labs to build the extraction, it's a separate scope with its own timeline (open question #14).

Topic 3: How does the orchestrator send suggestions back?

Question: What does the suggestion item need, and does the endpoint accept one target user or several? Artem will send the revised suggestion-item structure and endpoint for us to review.

Answer: We'll review Artem's revised structure and return changes. What we need in each item:

Field	Why
Target users (one or more)	We choose who receives it (topic 4)
Applicable role	Shown to the user; used for the owner fallback
Project ID	Every suggestion belongs to one project

Field	Why
Suggestion type	One of the actions Veda runs, from the activity type
Title and narrative	What the person reads
Action payload	The schema Veda already uses for that type
Source	Event sequence number, policy and revision, activity, methodology version
Evidence and confidence	Evidence references (call, ticket, health event) and a low, medium or high label
`expires_at`	Some suggestions stop being useful after a date
Deduplication key	A repeat is ignored, so retries are safe

From the endpoint we need:

1. **Several target users per suggestion:** one shared suggestion; the first person to apply it completes it for everyone (#23).
2. **A result per item:** created, duplicate or rejected, with a machine-readable error code; one bad item doesn't fail the rest.
3. **Outcome events in the feed, in the first version,** written in the same transaction as the change: `applied`, `dismissed`, `expired` (sent before cleanup deletes the suggestion), and the action's `succeeded` or `failed`. Each carries stable IDs, a timestamp, who acted, the request ID and an error code. An applied suggestion can still fail, so the two must be separate.
4. **Payload schemas** for each suggestion type, published in the catalog.

For observability we track feed lag, time from event to suggestion, duplicate and rejection rates, and counts of applied, dismissed and expired suggestions and of actions that succeeded or failed.

Topic 4: Who receives a suggestion?

Question: Is a suggestion addressed to a role, a specific user, or both? Who picks the user, and what happens when several people hold the same role?

Answer: The orchestrator picks the users; the suggestion names them and the role it applies to.

1. **Veda provides the team:** project ID, user ID, role or roles and assignment metadata for each member, in the context (topic 2).
2. **We choose:** the activity's role narrows the candidates; methodology, project context, current commitments and prior work on the project pick the most appropriate person.
3. **No reliable distinction:** we send one shared suggestion to everyone holding the role; the first to apply completes it for all (#23).
4. **Nobody holds the role:** it goes to the project owner, with the role kept so the UI can say no one holds it.
5. **Who can apply (#24):** we agree the person a suggestion is for can apply it whatever their access level, except stage moves, which need a project admin.
6. **Team changes:** when `project.team.changed` arrives, we re-check only pending suggestions. Either their recipients are updated, or the old one is retired and replaced in one step. There must never be two live copies that could, for example, send the same email twice.

Topic 5: Shared values and versioning

Question: Which shared lists and canonical values do both teams use (event, trigger, activity and suggestion types, status values and other integration fields), and how are events, methodology schemas, activity types and APIs versioned so changes don't break the integration?

Answer: One catalog in Veda, read through the catalog endpoint, is the source for every shared value:

- Keys are lowercase and never change once used; labels are display-only.
- Events and triggers are named `<domain>.<entity>.<event>`; reasons and status values use snake_case.
- The catalog includes status values (suggestion, outcome, methodology, project health), not just types.
- An unknown key is rejected with an error, never guessed.

Versioning:

- Each event type, the methodology schema, the activity-type list, the catalog and each API carries a version (`schema_version` in payloads; a version in the API path or header).
- Adding a value or an optional field is a minor change: readers ignore what they don't know. Our contracts already read this way.
- Renaming or removing a key, or making a field required, is a new major version. Both versions are served side by side until an agreed retirement date, and neither team changes this alone.
- The catalog has a version or hash, so we notice changes.

Time-based triggers: if a December use case needs one, Veda's scheduler adds one scheduled event per project to the feed (topic 1), and we evaluate date rules against the project's context. Moonnox owns that clock.

Final keys follow once the December use cases are agreed. **Certinia suggestion A, B and C:** the Architecture tab marks its mapping (stage update, create RAID, update RAID) as a guess; we'll confirm it with Certinia. **Insight/content creation and document generation** have no suggestion type in the first version; we'll check the December use cases don't need them.

Policies, activities and autonomy

Question: How do a policy's natural-language trigger and an explicit configured trigger coexist? How are activity descriptions and explicit agent or action assignments read? How is autonomy configured, and what's the difference between a suggestion and an autonomous action?

Answer:

- **Explicit and natural-language triggers.** The explicit trigger decides when a policy is looked at; the text decides whether it applies and what to suggest. A policy with only text is checked against the December triggers, not every event. If the text contradicts the configured trigger, the configured trigger wins and we flag the conflict to the admin.
- **Activities.** The activity type decides the suggestion type. An explicit agent or action assignment always wins over anything read from the description; the description and prompt shape the suggestion's content.

- **Suggestion vs autonomous action.** A suggestion is shown to a person, who applies or dismisses it; nothing happens until they apply it. An autonomous action is carried out without a person approving it.
- **Autonomy setting,** per activity with a policy-level default: always needs approval; may act autonomously; or needs approval when confidence or context is below a threshold.
- **December:** every action is a suggestion a person approves. An autonomous use case starts only when both teams approve it, with a confidence threshold, an audit record of every action, a way to undo it, and an off switch per tenant.

Topic 6: How do stage changes work?

Question: How are stage transitions represented, and can moving to the next stage itself be a suggestion, for example prompting a PM or seller to advance once the required activities are complete?

Answer: A project admin moves the stage; we only suggest the move, with `project.methodology.change_stage` and evidence, sent to the project owner and admins. Applying it runs the same step a person uses.

- **We'll build the stage-advance suggestion** once it's approved. It's sent when the evidence shows the stage's required activities are done, citing that evidence. A stage can also look complete from a call (for example, scope signed off).
- **Knowing an activity is done:** MOON-2389 defers activity tracking to a later phase. For December we count an activity as done when a suggestion for it was applied and its action succeeded (topic 3's outcome events). Activities done outside Veda won't be seen until activity tracking exists.
- **Stage-change events** say whether a person or an applied suggestion made the move, with the `suggestion_id`. When one arrives, we suggest the next stage's activities.
- The methodology stage and the Certinia stage stay separate, with no automatic sync.

Methodology versions and updates

Question: Clients will update methodology after project retrospectives, changing activities, triggers and policies. How do updates affect active projects and suggestions already made?

Answer:

- **Every suggestion records** the methodology version and policy revision it came from (topic 3's source field).
- **New evaluations** use the version the project is on: immediately after publishing, or after the project is upgraded, depending on open question #9.
- **Pending suggestions** whose activity or policy was removed or archived are retired; the rest stay valid until they expire.
- **Applied and dismissed suggestions** are history and never change.

Technical details: authentication and tracing

Question: How does the orchestrator authenticate to Veda? The meeting agreed Veda implements scoped service credentials in its key management (KMS), with rotation and secure storage, and provides a secure API so we retrieve credentials or authorization at runtime rather than from static configuration.

Answer: We agree. What we need from the design:

1. **Retrieval at runtime:** our service calls Veda's credentials API and receives a short-lived, scoped token (minutes, not days). No long-lived secret sits in our configuration.
2. **The first call:** that call needs an identity of its own. We propose our service's cloud workload identity (for example, an AWS IAM role, per environment) rather than a stored bootstrap secret. Veda confirms which identity it trusts.
3. **Scopes:** read the event feed, read context and methodology, write suggestions; each limited to the tenants we're allowed to see. Nothing else.
4. **Rotation and expiry:** keys rotate in Veda's KMS without action on our side; tokens expire and we fetch new ones before they do.
5. **Revocation and incidents:** either side can revoke our identity at once. We agree who is called, and how fast, if a credential may be exposed.
6. **Validation:** a test in the dev tenant shows every call we make (feed, context, methodology, catalog, suggestions) succeeds with the right scope and fails without it.
7. **Tracing:** we send `X-Request-Id` on every call and log it as our correlation ID.

This must be in place before any access is granted.

Open question #14: What should uploading a methodology document do?

Question: Beyond storing the file, should upload produce structured methodology, and who extracts it? The meeting made the upload experience Veda Accelerate's first priority, generating the structured file from an uploaded document.

Answer:

- **Upload produces the same structured JSON** as the interim file (see Methodology input), so the orchestrator reads uploaded and hand-built methodology the same way.
- **Who converts:** we propose Veda owns the conversion, as part of the upload experience. If Veda wants AI Labs to build the extraction, it's a separate scope with its own timeline; its result would be a proposal an admin accepts, with publishing a separate step.
- **Whether upload is in the first version** is Sa'ar's decision; the orchestrator works either way.
- **Later, not December:** a quality check at upload (ambiguity, contradictions, undefined terms), a comparison of activities against the agent registry to find activities with no agent, and project-specific agents for those gaps. We can contribute to all three.

Open question #5: Which values does each shared list have, and what are they called?

Question: What are the exact values and names for trigger types, trigger reasons, activity types (including Certinia suggestion A, B and C), suggestion types, event types, target kinds, source kinds, status values and methodology types?

Answer: As in topic 5: one versioned catalog in Veda, with the naming rules there. We'll confirm the final lists, any time-based trigger keys and the Certinia mapping once the December use cases are agreed.

Test data

Question: Will the MVP use live customer data, synthetic data, or a staged combination, and what approvals and controls apply?

Answer: A staged combination.

1. **Now:** synthetic data only, in a dev tenant.

2. **Live data** only after written approval from the customer, Certinia legal, privacy and security, and AI Labs. The outcome of the Monday call (led by David) decides whether and when.
3. **Where it runs:** live data stays in a Certinia- or Moonnox-hosted environment, never in AI Labs repositories, test fixtures or CI.
4. **Our code** keeps rejecting real data until the approval exists; the rule then changes through a reviewed change, not before.
5. **Controls to agree:** masking or anonymization, who can access it, retention, deletion at the end of the MVP, and which fields may be sent to Arthur for scoring.

For the joint working session

What we propose to close together, with an owner for each:

Item	Owner
The two or three December triggers and their delivery (topic 1)	Both teams
One event layer for the selected triggers	Both teams
Interim methodology file: format and delivery date	Veda Accelerate
Revised suggestion item and endpoint, several target users	Artem; AI Labs reviews
Context endpoint with team roles and methodology	Artem
Credentials API and the workload identity it trusts	Veda Accelerate
Autonomy setting and the suggestion vs action definitions	Product and architecture
Versioning rules for events, schemas and APIs	Both teams
Test data: outcome of the Monday call	David
Who converts uploaded documents (#14)	Artem